ASEN 3502 Computational Methods
Lab 5

Exact Due Date on Gradescope

General Instructions

1. You may not use any built-in optimization, curve-fitting, or machine-learning functions in your code unless a task explicitly permits it. In this lab, `fminsearch` is permitted in Week 2.
2. You may use any programming language, and you may use different languages for different parts. However, the primary goal of this lab is understanding, and *you should use Matlab unless every team member is enthusiastic about using something else.* **The least-squares function in Task 1.2 must be written in Matlab** so that another team can run it in the exchange.
3. Adhere to the AI policy at asen3502.com/ai-policy.
4. During each lab session, each student in the group should spend some time as the "driver".
5. **NEW: Document your code.** In this lab, *every function* should have documentation in the form of Matlab *help text*, also known as a docstring. See Task 1.4 for more information.

In this lab, you will use real GPS data recorded at the NIST station in Boulder, CO to learn regression in two settings: fitting a line to ionosphere data (Week 1), and finding a position from satellite signals when the model is nonlinear (Week 2). The ionosphere matters because it delays GPS signals by several meters, mostly changing the height you compute.

How GPS finds a position

Each GPS satellite broadcasts its own position and the time at which the signal left it. The receiver records when the signal arrived, and multiplying the travel time by the speed of light gives a distance to that satellite, called a *range*. The position problem is then geometric (see Figure 1):

- **One range** says the receiver is somewhere on a sphere centered on that satellite, with the range as its radius.
- **Two ranges** give two spheres, and the receiver must lie on both. Two spheres intersect in a circle.
- **Three ranges** give three spheres. The third sphere cuts the circle at two points. One of them is usually far from the Earth's surface, so the other is easy to identify as the receiver (points A and B in the figure).
- **More ranges** narrow the answer down to a single point. In real data every range contains noise, so the spheres never meet exactly at one point. With more ranges than unknowns we instead look for the position that best fits all of them, which is a regression problem.

Compiled 2026-11-13 07:48:08

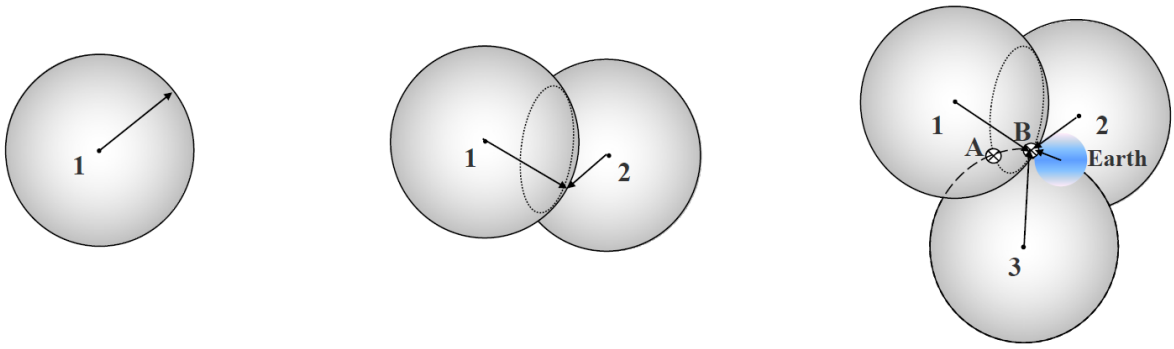


Figure 1: Finding a position from range measurements (trilateration). *Left:* one range to satellite 1 places the receiver on a sphere centered on that satellite. *Center:* a second range to satellite 2 restricts the receiver to the circle (dotted) where the two spheres intersect. *Right:* a third range to satellite 3 intersects that circle at two points, A and B; B is the one near the Earth's surface. Figure from ASEN 5090.

There is one complication. The receiver's clock is not perfectly synchronized with the satellites' clocks, so the travel time it measures is off by some unknown amount δt. Every range is then wrong by the same distance $b = c\,\delta t$, where c is the speed of light (a clock error of just 1 microsecond is about 300 m). Because the error is the same for all satellites, the effect on the picture above is that *all the sphere radii are too large (or too small) by the same amount* b, and the spheres no longer meet at the receiver. For this reason the measured distances are called *pseudoranges*. The clock offset b is unknown, but we do not need to know it ahead of time: it can be estimated together with the receiver position. That makes four unknowns (three position coordinates and b), so at least four satellites are needed. We have about nine per epoch, so the problem is overdetermined and we solve it by least squares.

Regression notation

Every fit in this lab uses the same standard notation. The data are pairs $(\vec{x}_i, y_i)$ for $i = 1, \ldots, n$. A *model* m with parameters $\vec{\theta}$ predicts each data point, and a *cost function* J, the *mean squared error*, averages the squared prediction errors:

$$\hat{y}_i = m(\vec{x}_i; \vec{\theta}), \qquad J(\vec{\theta}) = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \,. \tag{1}$$

Fitting the model means finding the $\vec{\theta}$ that minimizes J. When m is linear in $\vec{\theta}$ there is a closed-form answer (Week 1). When it is not, we minimize J numerically (Week 2).

The TEC model (Week 1). The ionosphere is described by the vertical total electron content (TEC). Over a window of a few hours we model it as a line in time:

$$\hat{y}_t = m_{\mathrm{TEC}}(t; \vec{\theta}_{\mathrm{TEC}}) = a_1 t + a_0, \qquad \vec{\theta}_{\mathrm{TEC}} = [a_0;\, a_1]. \tag{2}$$

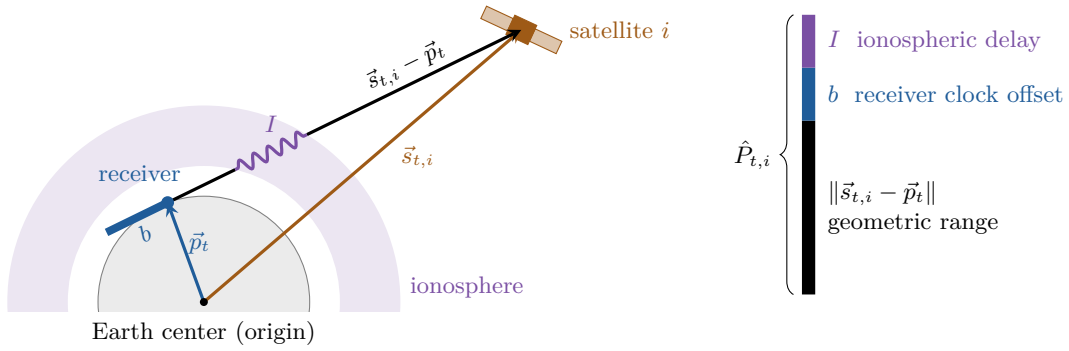


Figure 2: The pseudorange model for one satellite (not to scale). The satellite position $\vec{s}_{t,i}$ is known and the receiver position $\vec{p}_t$ is unknown. The receiver clock offset b makes the range appear longer, as if the receiver were farther back along the line of sight, and the ionosphere slows the signal, adding a delay I. The predicted pseudorange $\hat{P}_{t,i}$ is the sum of the geometric range and these two scalars, all in meters.

The GPS model (Week 2). At time t, satellite i at position $\vec{s}_{t,i}$ gives a measured pseudorange $P_{t,i}$. The model predicts it from the distance to the receiver position $\vec{p}_t$, the receiver clock offset b (in meters, as described above), and the ionospheric delay I:

$$\hat{P}_{t,i} = m_{\text{GPS}}(\vec{x}_{t,i}; \vec{\theta}_{\text{GPS},t}) = \|\vec{s}_{t,i} - \vec{p}_t\| + b + I\Big(\vec{s}_{t,i}, \vec{p}_t, m_{\text{TEC}}(t; \vec{\theta}_{\text{TEC}})\Big)\,, \tag{3}$$

(see Figure 2) with model input $\vec{x}_{t,i} = [\vec{s}_{t,i};\, t]$ and unknown parameters $\vec{\theta}_{\text{GPS},t} = [\vec{p}_t;\, b]$. The cost for epoch t is

$$J(\vec{\theta}_{\text{GPS},t}) = \frac{1}{n}\sum_{i=1}^{n}\left(P_{t,i} - \hat{P}_{t,i}\right)^2. \tag{4}$$

The delay I depends on the satellite position, the receiver position, and the TEC at that time. Task 1.7 builds a model for it. Throughout the lab, all positions, ranges, delays, and clock offsets are in **meters**.

The data

Your data set covers one full day (20 October 2026) of GPS measurements received by the NIST station in Boulder. The data stores one epoch every 5 minutes (288 epochs), with satellites below 10° elevation removed (there are about 9 satellites per epoch). The pseudoranges have already been corrected for many major errors: satellite clock, relativity, group delay, and the troposphere. After these corrections, each measurement is approximately equal to the geometric range between the satellite and receiver, plus the receiver clock offset, plus the ionospheric delay (plus noise). See Equation 3. You are given:

- `lab5_gnss.mat` (also `lab5_gnss_obs.csv`): a table `obs` with one row per satellite per epoch. Its columns are `Epoch`, `t_hr` (hours since midnight), `PRN`, `SatX`, `SatY`, `SatZ` (satellite position in ECEF, m), `El` and `Az` (degrees), and the corrected pseudoranges `C1C_corr` (the L1 C/A-code pseudorange, frequency f_1) and `C2W_corr` (the L2 P(Y)-code pseudorange, frequency f_2), both in m (only `C1C_corr` is needed in this lab). The file also holds `NIST` (the surveyed antenna position `p_true` in ECEF, plus latitude, longitude, and height) and `consts` (the speed of light c = 299,792,458 m/s and the GPS frequencies: L1 f_1 = 1575.42 MHz, L2 f_2 = 1227.60 MHz, and L5 f_5 = 1176.45 MHz; GPS transmits on all three, but in this lab we only use L1, f_1). Use `NIST.p_true` as the truth position for every GPS error calculation.
- `lab5_nist_vtec.mat` (also `lab5_nist_vtec.csv`): a struct `nist_vtec` with the vertical TEC (VTEC) at NIST from the GloTEC maps, interpolated between the four surrounding grid points: `t_hr` (hours

Compiled 2026-11-13 07:48:08

since midnight UTC) and `vtec` (TECU). Both are ordinary numeric vectors. One map is produced every 10 minutes. Use only this NIST series for all TEC work in this lab.

Provided starter functions. You will be given the following coordinate and geometry helpers (use meters and degrees unless stated otherwise):

- `p = lla2ecef_simple(lat, lon, h)`: geodetic latitude/longitude (degrees) and height (m) to a 3×1 ECEF position (m).
- `[lat, lon, h] = ecef2lla_simple(p)`: ECEF position (m) to geodetic latitude/longitude (degrees) and height (m).
- `E = enu_matrix(lat, lon)`: 3×3 rotation from ECEF differences to East-North-Up components at a geodetic latitude/longitude (degrees).
- `el_deg = elevation_deg(sats, p)`: elevations (degrees) for a satellite-position matrix (one ECEF row per satellite, m), viewed from receiver position p (ECEF, m).

You will implement a scalar linear least squares function, `iono_delay`, and `gps_cost` yourself. Use built-in plotting commands; no plotting helpers are required.

Deliverables (at end of Week 2)

1. To the **Lab 5 Writeup** assignment on Gradescope: A **single pdf** writeup that contains all of the items marked writeup.
   - Make sure to clearly label all plots and figures and use correct units where appropriate (TECU, meters, degrees, hours).
   - The lab writeup does *not* need to include narrative text unless specifically requested; it can just include the plot or results from each writeup task labeled with which task they are from.
2. To the **Lab 5 Code** assignment on Gradescope: Your **source code** for all code items.
   - Please help the TAs by clearly labelling where everything is, e.g. use filenames like `week_1.m`.
   - Unlike Lab 4, documentation counts for your least-squares function (see the General Instructions).
3. Tasks marked gradescope are completed directly on Gradescope during the lab, as described in the task.

1 Week 1: A Linear Model for TEC

1.1 Explore the data

writeup code

Load `lab5_nist_vtec.mat` and plot `vtec` against `t_hr` for the full day. Mark the fitting window by plotting vertical lines at 14 and 18 hr UTC (8 to 12 local time). Describe the daily shape of the curve (when it peaks, how quickly it rises, etc.) and explain why one straight line cannot describe the whole day but might describe the window.

1.2 Write a scalar linear regression function

In this task, you will write a scalar linear regression function in Matlab that will be shared and used by another team. The function should calculate the model parameters and the r^2 value for the fit. You are allowed to determine the name of the function and the input and output arguments. Make sure to:

- Name the function, arguments, and variables appropriately.
- Document your function with *help text* to describe how to use it according to the instructions at https://www.mathworks.com/help/matlab/matlab_prog/add-help-for-your-program.html. This documentation should include all information needed to use the function, including describing the format and meaning of input and output arguments.

- The users of your function are other students in the class, so use a simple style that they are likely to understand.
- Add comments *where appropriate* (in many programs there should be no comments other than the docstring).
- **You must include your team number in the help text so that the users know which team wrote it.**

1.3 Test your linear regression function

code

Before sharing your function, check it on small data sets where you know the answer. Write a short script that runs the following tests and compares the outputs to the expected values (allowing for round-off errors):

a. **Points exactly on a line.** Generate data from a line that you choose, e.g. `x = 0:4` and `y = 2*x + 1`. The fit should recover the parameters exactly ($a_0 = 1$, $a_1 = 2$) with $r^2 = 1$.
b. **Points not on a line.** For data points (0, 1), (1, 0), (2, 3), and (3,2), the fit should give $a_0 = 0.6$, $a_1 = 0.6$, and $r^2 = 0.36$.

If a test fails, fix the function (or the help text) before moving on.

1.4 Exchange code

gradescope

After all teammates have read the code and agree that it looks good, share it by uploading to gradescope. Upload the single file that contains your linear regression code to the gradescope assignment `Lab 5 Exchange`. Then notify the TAs and they will email code from another team. **Make sure to include all present team members in the gradescope submission group.**[1]

1.5 Fit the TEC model

writeup code

Use your least-squares function to fit m_{TEC} to the `vtec` data inside the 14 to 18 hr UTC window, using the time t in hours UTC as the input x and the VTEC as the output y. Report a_0 and a_1 with units. Plot the data for the whole day with the fitted line extended across the whole day, and report r^2 and the root-mean-square error (RMSE) for the points in the window and for all the points in the day. Interpret the two sets of numbers: where is the model good, and why is the whole-day r^2 so much lower (it can even be negative)?

1.6 Review the code

gradescope

After using the code to fit the TEC model, complete the `Lab 5 Review` gradescope assignment. **Make sure to include all present team members in the gradescope submission group.**[1]

1.7 Turn TEC into a delay

writeup code

A GPS signal crosses more of the ionosphere when the satellite is low in the sky, and the free electrons slow the signal down, so the measured range looks longer than the true range. We model the L1 delay on a satellite at elevation el as

$$I = K\,F(el)\,\text{TEC}_v, \qquad K = \frac{40.3 \times 10^{16}}{f_1^2}\ \text{m/TECU}, \qquad F(el) = \frac{1}{\sqrt{1 - \left(\dfrac{R_E \cos(el)}{R_E + h}\right)^2}}, \tag{5}$$

where TEC_v is the vertical TEC in TECU (1 TECU = 10^{16} electrons/m^2), $f_1 = 1575.42$ MHz is the L1 frequency (also stored in `consts`), $R_E = 6371$ km, and $h = 350$ km is the height of a thin shell that represents the ionosphere.

[1] As noted in the syllabus, there is not a penalty for a single absence from a lab.

a. Compute K (with f_1 in Hz) and check that it is about 0.16 m per TECU.
b. Write a function `I = iono_delay(el_deg, vtec)` that returns the delay in meters. Be sure to add clean documentation, as you will use this function in Week 2.
c. Using the VTEC from your fit, calculate and plot the ionospheric delay (in meters) against elevation angle (from 10° to 90°) at the start, middle, and end of the window. The plot should have 3 lines, one for each time: 14:00, 16:00, and 18:00 UTC.
d. How different is the delay at 10° from the delay at zenith (90°)? How does the delay change over the 4 hours?

2 Week 2: Nonlinear Regression for GPS Position

In Week 2 the model m_{GPS} is nonlinear in the receiver position $\vec{p}_t$, so there is no closed-form solution. We minimize $J(\vec{\theta}_{\text{GPS},t})$ numerically with `fminsearch`, which needs no gradient. We treat the clock offset b as a fourth unknown that `fminsearch` estimates along with the three position coordinates.

2.1 Write the GPS cost function

writeup code

Write a function `J = gps_cost(theta, sats, P, iono)` that evaluates the cost J for the four-element parameter vector $\vec{\theta}_{\text{GPS}} = [\vec{p}; b]$. The inputs are:

- `theta`: 4×1 vector $[x; y; z; b]$ with the trial ECEF position $\vec{p}$ and clock offset b, all in meters.
- `sats`: $n \times 3$ matrix of satellite ECEF positions (m).
- `P`: $n \times 1$ vector of measured pseudoranges (m).
- `iono`: a function handle with the interface `iono(sats, p)` that returns an $n \times 1$ vector of modeled delays (m) for the trial position `p` (ECEF, m). Use a handle that returns zeros for no correction, for example `@(sats,p) zeros(size(sats,1),1)`. The delay depends on the time of the epoch, so you need a different `iono` handle for each epoch. The easiest way is to write one function that does the complex delay calculation, then create a new anonymous function for each epoch that calls it with that epoch's values.

The function returns the mean squared error J in m^2. Document it. Because positions, pseudoranges, delays, and the clock offset are all in meters, no unit conversions are needed inside the function, and tolerances such as `TolX` are in meters.

2.2 Solve one epoch *without a correction*

writeup code

Take the epoch at 16:00 UTC and use the `C1C_corr` pseudoranges as they are, with no ionosphere correction, so $I = 0$ and the model is

$$\hat{P}_{t,i} = \|\vec{s}_{t,i} - \vec{p}_t\| + b.$$

Start from a position guess of 40°N, 105°W at zero height (convert to ECEF with the provided function) and a clock offset guess $b_0 = 0$, so that $\vec{\theta}_0 = [\vec{p}_0; 0]$. Call `fminsearch` with

```
optimset('TolX',1e-3,'TolFun',1e-8,'MaxFunEvals',5000,'MaxIter',2000).
```

Report the final East-North-Up error relative to `NIST.p_true`, the estimated clock offset b, the number of iterations and function evaluations, and the run time. Which component has the largest error? How far was your starting guess from the true position?

2.3 Correct with the TEC model

writeup code

Now put your Week 1 model into the GPS model. The delay on satellite i is

$$I\left(\vec{s}_{t,i}, \vec{p}_t, m_{\text{TEC}}(t; \vec{\theta}_{\text{TEC}})\right) = \texttt{iono_delay}\left(el_i(\vec{p}_t),\ m_{\text{TEC}}(t; \vec{\theta}_{\text{TEC}})\right),$$

where $el_i(\vec{p}_t)$ is the elevation of satellite i as seen from the trial position, so the delay is recomputed at every trial position. Use the supplied `elevation_deg` helper for this calculation, wrapped in a function handle that has the `iono(sats, p)` interface, for example `@(sats,p) iono_delay(elevation_deg(sats,p), tec)`. The elevation follows from the unit vector $\hat{\vec{u}}$ pointing up at $\vec{p}_t$:

$$el_i = \arcsin\left(\frac{\hat{\vec{u}} \cdot (\vec{s}_{t,i} - \vec{p}_t)}{\|\vec{s}_{t,i} - \vec{p}_t\|}\right).$$

a. Evaluate m_{TEC} at the epoch time using the linear fit from Task 1.5 (14–18 hr UTC).
b. Solve the same epoch with your cost function and this delay model, starting from the same $\vec{\theta}_0$ that worked in Task 2.2.
c. Make a table of the East, North, and Up errors and the 3D error for both solutions (no correction and TEC-corrected), each relative to `NIST.p_true`. Which component improves most? Does the correction improve all components?
d. Report the clock offset b for both solutions. Why does it change?
e. Convert the corrected solution from ECEF to latitude, longitude, and height with the provided `ecef2lla_simple`. Search the latitude and longitude in Google Maps to make sure your GPS solution is located at NIST.

2.4 Solve through the window

writeup code

Repeat both solutions (no correction and TEC-corrected) for every epoch from 14 to 18 hr UTC (where the linear TEC model applies), starting each solve from the same initial guess $\vec{\theta}_0$ (use the b_0 that worked in Task 2.2). At each epoch compute East, North, and Up errors relative to `NIST.p_true`. Plot the three error components against time, showing both methods on each plot, and give a table of RMS horizontal, vertical, and 3D errors for both methods. Finally, apply the TEC correction at one epoch well outside the window (for example 03:00 UTC) and describe what happens. What would you need to change to model the TEC reliably at that time?